```
import kagglehub
path = kagglehub.dataset_download("uciml/mushroom-classification")
import pandas as pd
import numpy as np

df = pd.read_csv("/content/mushrooms.csv")
df.head()
```

```
Using Colab cache for faster access to the 'mushroom-classification' dataset.
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk-surface-below-ring	stalk-color-above-ring	stalk-color-below-rin
0	p	x	s	n	t	p	f	c	n	k	...	s	w	w
1	e	x	s	y	t	a	f	c	b	k	...	s	w	w
2	e	b	s	w	t	l	f	c	b	n	...	s	w	w
3	p	x	y	w	t	p	f	c	n	n	...	s	w	w
4	e	x	s	g	f	n	f	w	b	k	...	s	w	w

5 rows × 23 columns

```
train_df, test_df = train_test_split(df, test_size=0.2, random_state=23)
print("Training samples:", len(train_df))
print("Testing samples:", len(test_df))


feature_prob = {}

for class_value, group_df in train_df.groupby("class"):
    feature_prob[class_value] = {}

    for column in train_df.columns:
        if column == "class":
            continue

        value_counts = group_df[column].value_counts()
        total = len(group_df[column])

        for value, count in value_counts.items():
            probability = count / total
            feature_prob[class_value][(column, value)] = probability

class_prob = {}
total_rows = len(train_df)

for class_value, count in train_df["class"].value_counts().items():
    class_prob[class_value] = count / total_rows

print("Class probabilities:", class_prob)


SMOOTHING_VALUE = 1e-9

correct_predictions = 0

y_true = test_df["class"]
x_test = test_df.drop(columns=["class"])

for row_index, row in x_test.iterrows():
    best_score = -1
    best_class = None

    for class_value in class_prob.keys():
        score = class_prob[class_value]

        for column_name, value in row.items():
            key = (column_name, value)
            score = score * feature_prob[class_value].get(key, SMOOTHING_VALUE)

        if score > best_score:
            best_score = score
            best_class = class_value
```

```
    if best_class == y_true[row_index]:
        correct_predictions += 1

accuracy = correct_predictions / len(y_true) * 100
print(f"Accuracy: {accuracy:.2f}%")
```

```
Training samples: 6499
Testing samples: 1625
Class probabilities: {'e': 0.5150023080473919, 'p': 0.4849976919526081}
Accuracy: 99.57%
```